Design and Implementation of a Console-Based E-Commerce System using Java

Assignment 1 — Report

Student name: Madiyar Seilkhan

Course: ICT

Date of submission: 29 October 2026

Git repository: https://github.com/madiyaka/ECommerceSystem

Contents

1. Objective
2. Requirement Analysis
3. Object Modeling
4. UML Class Diagram
5. Design and Implementation Notes
6. How to Run and Sample Session
7. Conclusion
8. Annex A — Source Code

1. Objective

The goal of this project is to design and implement a simple console-based e-commerce system in Java. The system manages users (customers and employees), products (books, electronics and clothing) and orders. Customers can browse products, fill a shopping cart and place orders; employees can add products and users, restock products and view all orders.

The project is used to practise the main principles of object-oriented programming: class design with attributes and methods, **encapsulation**, **inheritance**, **abstraction** and **polymorphism**, together with the Java Collections Framework (`ArrayList`) and console input with `Scanner`.

2. Requirement Analysis

2.1 Functional requirements

ID	Requirement	Who	Implemented in
FR1	Log in with email and password; show a menu that depends on the role	All users	`Store.login()` , `ConsoleApp.login()`
FR2	Register a new customer account	Guest	`ConsoleApp.registerCustomer()` , `Store.addUser()`
FR3	List available products with category-specific details	All	`Product.displayInfo()` , `ConsoleApp.listProducts()`
FR4	Add products (Book, Electronics, Clothing) to the system	Employee	`ConsoleApp.addProduct()` , `Store.addProduct()`
FR5	Add users (Customer or Employee) and list all users	Employee	`ConsoleApp.addUser()` , `ConsoleApp.listUsers()`
FR6	Add products to a cart and view the cart	Customer	`ConsoleApp.addToCart()` , `viewCart()`
FR7	Place an order from the cart; stock decreases; a receipt is printed	Customer	`Store.placeOrder()` , `Order.printReceipt()`

FR8	View own orders (customer) / all orders (employee)	Customer, Employee	`Store.getOrdersByCustomer()`, `Store.getOrders()`
FR9	Restock a product	Employee	`Product.increaseStock()`

2.2 Non-functional requirements

- All attributes are `private` and accessed through getters/setters (encapsulation).
- Data is kept in memory in `ArrayList` collections.
- Invalid input (letters instead of numbers, empty values, unknown IDs, too large quantities) must not crash the program.
- Code is organised in packages and uses meaningful names.

2.3 Identifying objects, attributes and methods

Classes were found by taking the **nouns** from the requirements (user, customer, employee, product, book, electronics, clothing, order) and methods from the **verbs** (add, list, place order, view, print receipt).

Object	Attributes	Main methods
User	userId, name, email, password, role	checkPassword(), displayInfo(), getters/setters
Customer	+ shippingAddress	getShippingAddress(), setShippingAddress()
Employee	+ position, salary	getPosition(), setSalary()
Product	id, name, price, stockQuantity	displayInfo(), isInStock(), reduceStock(), increaseStock()
Book	+ author, pages	getCategory(), getDetails()
Electronics	+ brand, warrantyMonths	getCategory(), getDetails()
Clothing	+ size, color	getCategory(), getDetails()
Order	orderId, customer, items, totalAmount, orderDate	addItem(), calculateTotal(), printReceipt()
OrderItem	product, quantity, unitPrice	getSubtotal()
Store	users, products, orders (ArrayLists), ID counters	addUser(), addProduct(), login(), placeOrder(), find...()

`OrderItem` and `Store` are not named directly in the requirements; they were added during design (see section 5.5).

3. Object Modeling

3.1 Classes and packages

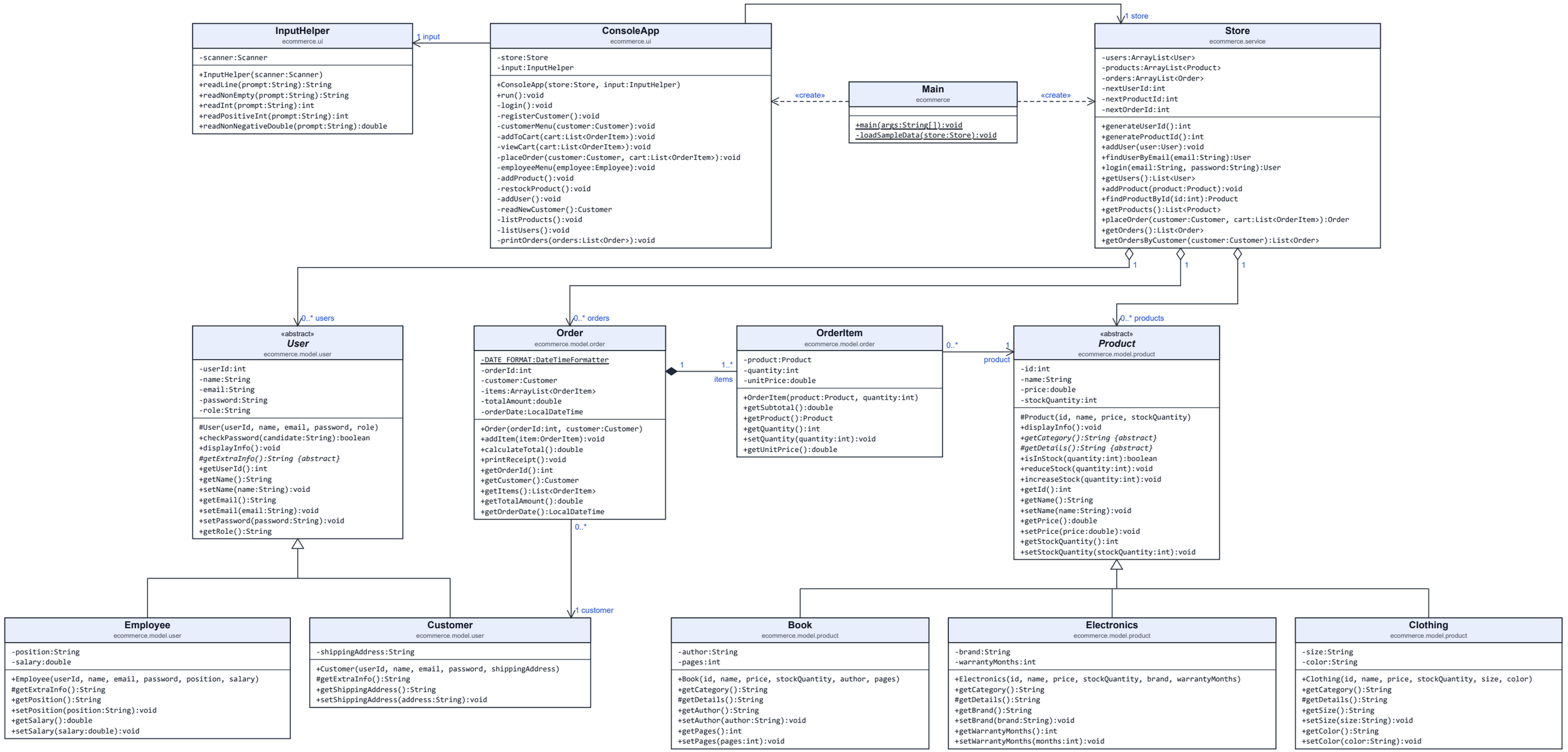
Package	Classes	Responsibility
`ecommerce`	Main	Program entry point, loads demo data
`ecommerce.model.user`	User (abstract), Customer, Employee	People who use the system

`ecommerce.model.product`	Product (abstract), Book, Electronics, Clothing	Items that are sold
`ecommerce.model.order`	Order, OrderItem	Purchases
`ecommerce.service`	Store	Stores all data in ArrayLists and applies business rules
`ecommerce.ui`	ConsoleApp, InputHelper	Console menus and safe input with Scanner

3.2 Relationships

Relationship	Type	Multiplicity	Meaning
Customer, Employee → User	Inheritance (generalization)	–	A customer *is a* user; an employee *is a* user
Book, Electronics, Clothing → Product	Inheritance (generalization)	–	Each is a kind of product
Order → Customer	Association	0..* orders – 1 customer	Every order belongs to exactly one customer
Order ◆– OrderItem	Composition	1 – 1..*	Order items exist only as part of an order
OrderItem → Product	Association	0..* – 1	Each line refers to one product
Store ◇– User / Product / Order	Aggregation	1 – 0..*	The store keeps collections of all users, products and orders
ConsoleApp → Store, InputHelper	Association	1 – 1	The UI uses the store for data and InputHelper for reading input
Main ..> ConsoleApp, Store	Dependency «create»	–	Main creates the objects and starts the app

4. UML Class Diagram



Notation: **-** private, **+** public, **#** protected; *italic* = abstract class/method; underlined = static. Hollow triangle = inheritance, filled diamond = composition, hollow diamond = aggregation, open arrow = navigable association, dashed arrow = dependency. The diagram is also provided as `docs/uml-class-diagram.svg` / `.png` .

5. Design and Implementation Notes

5.1 Encapsulation

Every attribute in every class is `private`. Other classes access data only through public getters and setters. Setters are used to protect the object's state, for example `Product.setPrice()` and `setStockQuantity()` reject negative values, and `Employee.setSalary()` rejects a negative salary. The password has a setter but **no getter**: it can only be checked with `checkPassword()`, so it is never exposed. Methods that return lists (`Store.getProducts()`, `Order.getItems()`, ...) return a **copy** of the internal ArrayList, so code outside the class cannot add or remove elements behind the class's back. Stock can only change through `reduceStock()` / `increaseStock()`, which check the values.

5.2 Inheritance and abstraction

`User` and `Product` are declared `abstract` because a "plain" user or product never exists — there is always a concrete type. The common attributes (id, name, price, stock...) and common behaviour (`displayInfo()`, `isInStock()`, `checkPassword()`) are written once in the superclass and reused by all subclasses. Subclass constructors call `super(...)` to initialise the inherited private fields. The role of a user ("CUSTOMER" / "EMPLOYEE") is set automatically by the subclass constructor, so it can never be inconsistent with the real type.

5.3 Polymorphism

`Product.displayInfo()` prints the common fields and then calls the abstract methods `getCategory()` and `getDetails()`. Each subclass overrides them (`@Override`), so when the store loops over an `ArrayList<Product>` and calls `displayInfo()`, a book shows its author, electronics show brand and warranty, and clothing shows size and color. The same idea is used in `User.displayInfo()` with `getExtraInfo()`. After login, `instanceof` is used to decide whether the customer menu or the employee menu is shown.

5.4 Collections

`Store` keeps three collections: `ArrayList<User>`, `ArrayList<Product>` and `ArrayList<Order>`. Because the lists use the superclass types, one list can hold customers and employees (or books, electronics and clothing) together. `Order` keeps an `ArrayList<OrderItem>`, and the customer's shopping cart is a `List<OrderItem>` that lives only during the customer's session.

5.5 Design decisions

- **OrderItem class.** The requirements say an order contains a "list of products". A plain `ArrayList<Product>` cannot store *how many* units were bought, and if the product's price changed later, old receipts would change too. `OrderItem` stores the product, the quantity and the unit price at the time of purchase.
- **Store class (service layer).** All data and business rules are in `Store`; the menu code in `ConsoleApp` only reads input and prints output. This separation makes the code easier to read and would let us replace the console with a GUI without changing the model.

- **Safe order placement.** `Store.placeOrder()` first checks the stock of *every* cart item and only then reduces stock and creates the order, so an order is either completed fully or not at all.
- **Total amount consistency.** `Order.totalAmount` has no setter; it is recalculated by `calculateTotal()` every time an item is added, so it always matches the items.
- **Unique e-mail.** `Store.addUser()` refuses a second user with the same e-mail, because e-mail is used to log in.
- **InputHelper.** All `Scanner` input goes through this class, which repeats the question until the value is valid, so typing letters instead of a number does not throw an exception.
- **ID generation.** The store generates IDs with counters (`nextUserId` , ...), so IDs are unique and the user does not need to type them.

5.6 Console menus

Main menu	Customer menu	Employee menu
1. Login 2. Register as customer 3. Browse products (guest) 0. Exit	1. List products 2. Add product to cart 3. View cart 4. Place order 5. View my orders 0. Logout	1. List products 2. Add product 3. Restock product 4. List users 5. Add user 6. View all orders 0. Logout

6. How to Run and Sample Session

Open the project folder in IntelliJ IDEA (mark `src` as the sources root) and run `ecommerce.Main`, or from the command line:

```
javac -d out src/ecommerce/*.java src/ecommerce/*/*.java src/ecommerce/*/*/*.java
java -cp out ecommerce.Main
```

Demo accounts loaded at start-up:

Role	Email	Password
Employee (Manager)	admin@shop.kz	admin123
Customer	aida@mail.kz	1234
Customer	dan@mail.kz	1234

Part of a real session (customer lists products, adds a mouse and a laptop to the cart and places the order):

```
===== CUSTOMER MENU (Aida) =====
1. List products
2. Add product to cart
3. View cart
4. Place order
5. View my orders
0. Logout
Choose: 1
--- Products ---
#1   Book         Clean Code                    12500,00 KZT  stock: 10   author: Robert C. Martin,
464 pages
#3   Electronics  Wireless Mouse                 8900,00 KZT  stock: 20   brand: Logitech, warranty
12 months
#5   Clothing     Hoodie                        18000,00 KZT  stock: 15   size: M, color: Black
...
Choose: 4
Order placed successfully!
================================================
 RECEIPT  -  Order #1
 Date:     2026-10-29 23:02
 Customer: Aida (aida@mail.kz)
 Ship to:  Almaty, Abay Ave 10
------------------------------------------------
 Wireless Mouse             2 x   8900,00 =   17800,00
 Laptop 15"                 1 x 420000,00 =  420000,00
------------------------------------------------
 TOTAL:                              437800,00 KZT
================================================
```

Tested scenarios: login with wrong password; registration with an existing e-mail; entering letters instead of a number; adding more items than in stock (the program answers "Only 15 in stock."); placing an order with an empty cart; employee adding a new book and viewing all orders.

7. Conclusion

A working console e-commerce system was designed and implemented in Java. The requirement analysis produced ten classes in five packages. Two inheritance hierarchies (`User` → `Customer` / `Employee` and `Product` → `Book` / `Electronics` / `Clothing`) remove duplicated code and, together with abstract methods, give polymorphic behaviour when products and users are listed. Encapsulation with private fields and validating setters keeps every object in a valid state, and `ArrayList` collections store all data. Separating the model, the service (`Store`) and the user interface (`ConsoleApp`) keeps the code clear and easy to extend — for example, adding a new product category only requires one new subclass of `Product` .

Possible improvements: saving data to files or a database, hashing passwords, searching/filtering products, and order statuses (paid, shipped, delivered).

Annex A — Source Code

Git repository: **https://github.com/madiyaka/ECommerceSystem**

The complete source code of the project is also included below.

Project structure:

```
ECommerceSystem/
└── src/ecommerce/
    ├── Main.java
    ├── model/
    │   ├── user/     User.java, Customer.java, Employee.java
    │   ├── product/  Product.java, Book.java, Electronics.java, Clothing.java
    │   └── order/    Order.java, OrderItem.java
    ├── service/      Store.java
    └── ui/           ConsoleApp.java, InputHelper.java
```

A.1 src/ecommerce/Main.java

```
package ecommerce;

import ecommerce.model.product.Book;
import ecommerce.model.product.Clothing;
import ecommerce.model.product.Electronics;
import ecommerce.model.user.Customer;
import ecommerce.model.user.Employee;
import ecommerce.service.Store;
import ecommerce.ui.ConsoleApp;
import ecommerce.ui.InputHelper;

import java.util.Scanner;

/**
 * Entry point. Creates the store, loads demo data and starts the console menu.
 */
public class Main {

    public static void main(String[] args) {
        Store store = new Store();
        loadSampleData(store);

        Scanner scanner = new Scanner(System.in);
        ConsoleApp app = new ConsoleApp(store, new InputHelper(scanner));
        app.run();
        scanner.close();
    }

    /** Demo data so the application can be shown immediately. */
    private static void loadSampleData(Store store) {
        store.addUser(new Employee(store.generateUserId(), "Admin", "admin@shop.kz", "admin123",
"Manager", 450000));
        store.addUser(new Customer(store.generateUserId(), "Aida", "aida@mail.kz", "1234", "Almaty,
Abay Ave 10"));
        store.addUser(new Customer(store.generateUserId(), "Daniyar", "dan@mail.kz", "1234", "Astana,
Mangilik El 5"));

        store.addProduct(new Book(store.generateProductId(), "Clean Code", 12500, 10, "Robert C.
Martin", 464));
        store.addProduct(new Book(store.generateProductId(), "Head First Java", 15000, 5, "Kathy
```

```
Sierra", 720));
        store.addProduct(new Electronics(store.generateProductId(), "Wireless Mouse", 8900, 20,
"Logitech", 12));
        store.addProduct(new Electronics(store.generateProductId(), "Laptop 15\"", 420000, 3,
"Lenovo", 24));
        store.addProduct(new Clothing(store.generateProductId(), "Hoodie", 18000, 15, "M", "Black"));
        store.addProduct(new Clothing(store.generateProductId(), "T-Shirt", 6500, 30, "L", "White"));
    }
}
```

A.2 src/ecommerce/model/user/User.java

```
package ecommerce.model.user;

/**
 * Base class for every person who can log in to the system.
 * It is abstract because a "plain" user never exists: every user is
 * either a Customer or an Employee.
 */
public abstract class User {

    private int userId;
    private String name;
    private String email;
    private String password;
    private String role;

    protected User(int userId, String name, String email, String password, String role) {
        this.userId = userId;
        this.name = name;
        this.email = email;
        this.password = password;
        this.role = role;
    }

    /** Compares the given password with the stored one (password is never exposed by a getter). */
    public boolean checkPassword(String candidate) {
        return password.equals(candidate);
    }

    /** Prints a one-line summary of the user. Subclasses add their own details. */
    public void displayInfo() {
        System.out.printf("#%-3d %-10s %-20s %-25s%s%n",
                userId, role, name, email, getExtraInfo());
    }

    /** Hook for subclasses to append role-specific information to displayInfo(). */
    protected abstract String getExtraInfo();

    public int getUserId() {
        return userId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }
```

```
    public void setEmail(String email) {
        this.email = email;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }
}
```

A.3 src/ecommerce/model/user/Customer.java

```
package ecommerce.model.user;

/**
 * A customer can browse products and place orders.
 */
public class Customer extends User {

    private String shippingAddress;

    public Customer(int userId, String name, String email, String password, String shippingAddress) {
        super(userId, name, email, password, "CUSTOMER");
        this.shippingAddress = shippingAddress;
    }

    @Override
    protected String getExtraInfo() {
        return "address: " + shippingAddress;
    }

    public String getShippingAddress() {
        return shippingAddress;
    }

    public void setShippingAddress(String shippingAddress) {
        this.shippingAddress = shippingAddress;
    }
}
```

A.4 src/ecommerce/model/user/Employee.java

```
package ecommerce.model.user;

/**
 * An employee manages the store: adds products and users, restocks and views all orders.
 */
public class Employee extends User {

    private String position;
    private double salary;

    public Employee(int userId, String name, String email, String password, String position, double
salary) {
        super(userId, name, email, password, "EMPLOYEE");
        this.position = position;
        setSalary(salary);
    }

    @Override
    protected String getExtraInfo() {
```

```
        return "position: " + position;
    }

    public String getPosition() {
        return position;
    }

    public void setPosition(String position) {
        this.position = position;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        if (salary < 0) {
            throw new IllegalArgumentException("Salary cannot be negative");
        }
        this.salary = salary;
    }
}
```

A.5 src/ecommerce/model/product/Product.java

```
package ecommerce.model.product;

/**
 * Base class for everything the store sells.
 * Abstract because every real product belongs to a category (Book, Electronics, Clothing).
 */
public abstract class Product {

    private int id;
    private String name;
    private double price;
    private int stockQuantity;

    protected Product(int id, String name, double price, int stockQuantity) {
        this.id = id;
        this.name = name;
        setPrice(price);
        setStockQuantity(stockQuantity);
    }

    /** Prints the common product fields followed by the category-specific details. */
    public void displayInfo() {
        System.out.printf("#%-3d %-12s %-28s %10.2f KZT  stock: %-4d %s%n",
                id, getCategory(), name, price, stockQuantity, getDetails());
    }

    /** Category name shown in listings, e.g. "Book". */
    public abstract String getCategory();

    /** Category-specific attributes as text, e.g. "author: ...". */
    protected abstract String getDetails();

    public boolean isInStock(int quantity) {
        return quantity > 0 && quantity <= stockQuantity;
    }

    /** Removes sold items from stock. Called only after isInStock() was checked. */
    public void reduceStock(int quantity) {
        if (!isInStock(quantity)) {
            throw new IllegalArgumentException("Not enough stock for " + name);
```

```
        }
        stockQuantity -= quantity;
    }

    public void increaseStock(int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException("Quantity must be positive");
        }
        stockQuantity += quantity;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        if (price < 0) {
            throw new IllegalArgumentException("Price cannot be negative");
        }
        this.price = price;
    }

    public int getStockQuantity() {
        return stockQuantity;
    }

    public void setStockQuantity(int stockQuantity) {
        if (stockQuantity < 0) {
            throw new IllegalArgumentException("Stock cannot be negative");
        }
        this.stockQuantity = stockQuantity;
    }
}
```

A.6 src/ecommerce/model/product/Book.java

```
package ecommerce.model.product;

public class Book extends Product {

    private String author;
    private int pages;

    public Book(int id, String name, double price, int stockQuantity, String author, int pages) {
        super(id, name, price, stockQuantity);
        this.author = author;
        this.pages = pages;
    }

    @Override
    public String getCategory() {
        return "Book";
    }
```

```
    @Override
    protected String getDetails() {
        return "author: " + author + ", " + pages + " pages";
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public int getPages() {
        return pages;
    }

    public void setPages(int pages) {
        this.pages = pages;
    }
}
```

A.7 src/ecommerce/model/product/Electronics.java

```
package ecommerce.model.product;

public class Electronics extends Product {

    private String brand;
    private int warrantyMonths;

    public Electronics(int id, String name, double price, int stockQuantity, String brand, int
warrantyMonths) {
        super(id, name, price, stockQuantity);
        this.brand = brand;
        this.warrantyMonths = warrantyMonths;
    }

    @Override
    public String getCategory() {
        return "Electronics";
    }

    @Override
    protected String getDetails() {
        return "brand: " + brand + ", warranty " + warrantyMonths + " months";
    }

    public String getBrand() {
        return brand;
    }

    public void setBrand(String brand) {
        this.brand = brand;
    }

    public int getWarrantyMonths() {
        return warrantyMonths;
    }

    public void setWarrantyMonths(int warrantyMonths) {
        this.warrantyMonths = warrantyMonths;
    }
}
```

A.8 src/ecommerce/model/product/Clothing.java

```
package ecommerce.model.product;

public class Clothing extends Product {

    private String size;
    private String color;

    public Clothing(int id, String name, double price, int stockQuantity, String size, String color) {
        super(id, name, price, stockQuantity);
        this.size = size;
        this.color = color;
    }

    @Override
    public String getCategory() {
        return "Clothing";
    }

    @Override
    protected String getDetails() {
        return "size: " + size + ", color: " + color;
    }

    public String getSize() {
        return size;
    }

    public void setSize(String size) {
        this.size = size;
    }

    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }
}
```

A.9 src/ecommerce/model/order/Order.java

```
package ecommerce.model.order;

import ecommerce.model.user.Customer;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;

/**
 * An order placed by one customer, containing one or more order items.
 */
public class Order {

    private static final DateTimeFormatter DATE_FORMAT = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm");

    private int orderId;
    private Customer customer;
    private ArrayList<OrderItem> items;
```

```
    private double totalAmount;
    private LocalDateTime orderDate;

    public Order(int orderId, Customer customer) {
        this.orderId = orderId;
        this.customer = customer;
        this.items = new ArrayList<>();
        this.totalAmount = 0;
        this.orderDate = LocalDateTime.now();
    }

    public void addItem(OrderItem item) {
        items.add(item);
        calculateTotal();
    }

    /** Recalculates totalAmount from the items, so the total is always consistent. */
    public double calculateTotal() {
        double sum = 0;
        for (OrderItem item : items) {
            sum += item.getSubtotal();
        }
        totalAmount = sum;
        return totalAmount;
    }

    public void printReceipt() {
        System.out.println("================================================");
        System.out.println(" RECEIPT  -  Order #" + orderId);
        System.out.println(" Date:     " + orderDate.format(DATE_FORMAT));
        System.out.println(" Customer: " + customer.getName() + " (" + customer.getEmail() + ")");
        System.out.println(" Ship to:  " + customer.getShippingAddress());
        System.out.println("------------------------------------------------");
        for (OrderItem item : items) {
            System.out.printf(" %-24s %3d x %9.2f = %10.2f%n",
                    item.getProduct().getName(), item.getQuantity(),
                    item.getUnitPrice(), item.getSubtotal());
        }
        System.out.println("------------------------------------------------");
        System.out.printf(" TOTAL: %38.2f KZT%n", totalAmount);
        System.out.println("================================================");
    }

    public int getOrderId() {
        return orderId;
    }

    public Customer getCustomer() {
        return customer;
    }

    /** Returns a copy so callers cannot modify the order's items directly. */
    public List<OrderItem> getItems() {
        return new ArrayList<>(items);
    }

    public double getTotalAmount() {
        return totalAmount;
    }

    public LocalDateTime getOrderDate() {
        return orderDate;
    }
}
```

A.10 src/ecommerce/model/order/OrderItem.java

```
package ecommerce.model.order;

import ecommerce.model.product.Product;

/**
 * One line of an order: a product and how many units were bought.
 * The unit price is copied at purchase time so later price changes
 * do not change old receipts.
 */
public class OrderItem {

    private Product product;
    private int quantity;
    private double unitPrice;

    public OrderItem(Product product, int quantity) {
        this.product = product;
        this.quantity = quantity;
        this.unitPrice = product.getPrice();
    }

    public double getSubtotal() {
        return unitPrice * quantity;
    }

    public Product getProduct() {
        return product;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }

    public double getUnitPrice() {
        return unitPrice;
    }
}
```

A.11 src/ecommerce/service/Store.java

```
package ecommerce.service;

import ecommerce.model.order.Order;
import ecommerce.model.order.OrderItem;
import ecommerce.model.product.Product;
import ecommerce.model.user.Customer;
import ecommerce.model.user.User;

import java.util.ArrayList;
import java.util.List;

/**
 * Holds all data of the shop in ArrayLists and contains the business rules
 * (login, adding users/products, placing orders). The console UI only talks to this class.
 */
public class Store {

    private ArrayList<User> users = new ArrayList<>();
    private ArrayList<Product> products = new ArrayList<>();
    private ArrayList<Order> orders = new ArrayList<>();
```

```
    private int nextUserId = 1;
    private int nextProductId = 1;
    private int nextOrderId = 1;

    // ---------- ID generation ----------

    public int generateUserId() {
        return nextUserId++;
    }

    public int generateProductId() {
        return nextProductId++;
    }

    // ---------- Users ----------

    public void addUser(User user) {
        if (findUserByEmail(user.getEmail()) != null) {
            throw new IllegalArgumentException("A user with email " + user.getEmail() + " already
exists");
        }
        users.add(user);
    }

    public User findUserByEmail(String email) {
        for (User user : users) {
            if (user.getEmail().equalsIgnoreCase(email)) {
                return user;
            }
        }
        return null;
    }

    /** Returns the matching user, or null if the email/password is wrong. */
    public User login(String email, String password) {
        User user = findUserByEmail(email);
        if (user != null && user.checkPassword(password)) {
            return user;
        }
        return null;
    }

    public List<User> getUsers() {
        return new ArrayList<>(users);
    }

    // ---------- Products ----------

    public void addProduct(Product product) {
        products.add(product);
    }

    public Product findProductById(int id) {
        for (Product product : products) {
            if (product.getId() == id) {
                return product;
            }
        }
        return null;
    }

    public List<Product> getProducts() {
        return new ArrayList<>(products);
    }

    // ---------- Orders ----------
```

```
    /**
     * Creates an order from the customer's cart.
     * Stock is checked for every item first, so either the whole order succeeds or nothing changes.
     */
    public Order placeOrder(Customer customer, List<OrderItem> cart) {
        if (cart.isEmpty()) {
            throw new IllegalArgumentException("Cart is empty");
        }
        for (OrderItem item : cart) {
            if (!item.getProduct().isInStock(item.getQuantity())) {
                throw new IllegalArgumentException("Not enough stock for " +
item.getProduct().getName()
                        + " (available: " + item.getProduct().getStockQuantity() + ")");
            }
        }

        Order order = new Order(nextOrderId++, customer);
        for (OrderItem item : cart) {
            item.getProduct().reduceStock(item.getQuantity());
            order.addItem(item);
        }
        orders.add(order);
        return order;
    }

    public List<Order> getOrders() {
        return new ArrayList<>(orders);
    }

    public List<Order> getOrdersByCustomer(Customer customer) {
        List<Order> result = new ArrayList<>();
        for (Order order : orders) {
            if (order.getCustomer() == customer) {
                result.add(order);
            }
        }
        return result;
    }
}
```

A.12 src/ecommerce/ui/InputHelper.java

```
package ecommerce.ui;

import java.util.Scanner;

/**
 * Small wrapper around Scanner that keeps asking until the input is valid,
 * so a typo never crashes the program.
 */
public class InputHelper {

    private Scanner scanner;

    public InputHelper(Scanner scanner) {
        this.scanner = scanner;
    }

    public String readLine(String prompt) {
        System.out.print(prompt);
        return scanner.nextLine().trim();
    }

    public String readNonEmpty(String prompt) {
        while (true) {
```

```
            String value = readLine(prompt);
            if (!value.isEmpty()) {
                return value;
            }
            System.out.println("  Value cannot be empty.");
        }
    }

    public int readInt(String prompt) {
        while (true) {
            String value = readLine(prompt);
            try {
                return Integer.parseInt(value);
            } catch (NumberFormatException e) {
                System.out.println("  Please enter a whole number.");
            }
        }
    }

    public int readPositiveInt(String prompt) {
        while (true) {
            int value = readInt(prompt);
            if (value > 0) {
                return value;
            }
            System.out.println("  Number must be greater than 0.");
        }
    }

    public double readNonNegativeDouble(String prompt) {
        while (true) {
            String value = readLine(prompt);
            try {
                double number = Double.parseDouble(value);
                if (number >= 0) {
                    return number;
                }
                System.out.println("  Number cannot be negative.");
            } catch (NumberFormatException e) {
                System.out.println("  Please enter a number (e.g. 1500 or 99.90).");
            }
        }
    }
}
```

A.13 src/ecommerce/ui/ConsoleApp.java

```
package ecommerce.ui;

import ecommerce.model.order.Order;
import ecommerce.model.order.OrderItem;
import ecommerce.model.product.Book;
import ecommerce.model.product.Clothing;
import ecommerce.model.product.Electronics;
import ecommerce.model.product.Product;
import ecommerce.model.user.Customer;
import ecommerce.model.user.Employee;
import ecommerce.model.user.User;
import ecommerce.service.Store;

import java.util.ArrayList;
import java.util.List;

/**
 * Console user interface: main menu, customer menu and employee menu.
```

```
 * All data operations are delegated to the Store.
 */
public class ConsoleApp {

    private Store store;
    private InputHelper input;

    public ConsoleApp(Store store, InputHelper input) {
        this.store = store;
        this.input = input;
    }

    public void run() {
        System.out.println("Welcome to the Console E-Commerce System!");
        boolean running = true;
        while (running) {
            System.out.println();
            System.out.println("===== MAIN MENU =====");
            System.out.println("1. Login");
            System.out.println("2. Register as customer");
            System.out.println("3. Browse products (guest)");
            System.out.println("0. Exit");
            int choice = input.readInt("Choose: ");
            switch (choice) {
                case 1:
                    login();
                    break;
                case 2:
                    registerCustomer();
                    break;
                case 3:
                    listProducts();
                    break;
                case 0:
                    running = false;
                    break;
                default:
                    System.out.println("Unknown option.");
            }
        }
        System.out.println("Goodbye!");
    }

    private void login() {
        String email = input.readNonEmpty("Email: ");
        String password = input.readNonEmpty("Password: ");
        User user = store.login(email, password);
        if (user == null) {
            System.out.println("Wrong email or password.");
            return;
        }
        System.out.println("Hello, " + user.getName() + "! You are logged in as " + user.getRole() +
".");
        // The menu depends on the real (runtime) type of the user object.
        if (user instanceof Employee) {
            employeeMenu((Employee) user);
        } else if (user instanceof Customer) {
            customerMenu((Customer) user);
        }
    }

    private void registerCustomer() {
        try {
            store.addUser(readNewCustomer());
            System.out.println("Registration successful. You can now log in.");
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
```

```
        }
    }

    // ==================== CUSTOMER MENU ====================

    private void customerMenu(Customer customer) {
        List<OrderItem> cart = new ArrayList<>();
        boolean loggedIn = true;
        while (loggedIn) {
            System.out.println();
            System.out.println("===== CUSTOMER MENU (" + customer.getName() + ") =====");
            System.out.println("1. List products");
            System.out.println("2. Add product to cart");
            System.out.println("3. View cart");
            System.out.println("4. Place order");
            System.out.println("5. View my orders");
            System.out.println("0. Logout");
            int choice = input.readInt("Choose: ");
            switch (choice) {
                case 1:
                    listProducts();
                    break;
                case 2:
                    addToCart(cart);
                    break;
                case 3:
                    viewCart(cart);
                    break;
                case 4:
                    placeOrder(customer, cart);
                    break;
                case 5:
                    printOrders(store.getOrdersByCustomer(customer));
                    break;
                case 0:
                    loggedIn = false;
                    break;
                default:
                    System.out.println("Unknown option.");
            }
        }
    }

    private void addToCart(List<OrderItem> cart) {
        listProducts();
        int id = input.readInt("Product ID: ");
        Product product = store.findProductById(id);
        if (product == null) {
            System.out.println("No product with ID " + id + ".");
            return;
        }
        int quantity = input.readPositiveInt("Quantity: ");

        // If the product is already in the cart, increase its quantity instead of adding a second
line.
        OrderItem existing = null;
        for (OrderItem item : cart) {
            if (item.getProduct() == product) {
                existing = item;
            }
        }
        int newQuantity = (existing == null) ? quantity : existing.getQuantity() + quantity;
        if (!product.isInStock(newQuantity)) {
            System.out.println("Only " + product.getStockQuantity() + " in stock.");
            return;
        }
        if (existing == null) {
```

```
            cart.add(new OrderItem(product, quantity));
        } else {
            existing.setQuantity(newQuantity);
        }
        System.out.println("Added " + quantity + " x " + product.getName() + " to cart.");
    }

    private void viewCart(List<OrderItem> cart) {
        if (cart.isEmpty()) {
            System.out.println("Your cart is empty.");
            return;
        }
        double total = 0;
        System.out.println("--- Cart ---");
        for (OrderItem item : cart) {
            System.out.printf(" %-24s %3d x %9.2f = %10.2f%n",
                    item.getProduct().getName(), item.getQuantity(),
                    item.getUnitPrice(), item.getSubtotal());
            total += item.getSubtotal();
        }
        System.out.printf(" Total: %.2f KZT%n", total);
    }

    private void placeOrder(Customer customer, List<OrderItem> cart) {
        try {
            Order order = store.placeOrder(customer, cart);
            System.out.println("Order placed successfully!");
            order.printReceipt();
            cart.clear();
        } catch (IllegalArgumentException e) {
            System.out.println("Could not place order: " + e.getMessage());
        }
    }

    // ==================== EMPLOYEE MENU ====================

    private void employeeMenu(Employee employee) {
        boolean loggedIn = true;
        while (loggedIn) {
            System.out.println();
            System.out.println("===== EMPLOYEE MENU (" + employee.getName() + ", " +
employee.getPosition() + ") =====");
            System.out.println("1. List products");
            System.out.println("2. Add product");
            System.out.println("3. Restock product");
            System.out.println("4. List users");
            System.out.println("5. Add user");
            System.out.println("6. View all orders");
            System.out.println("0. Logout");
            int choice = input.readInt("Choose: ");
            switch (choice) {
                case 1:
                    listProducts();
                    break;
                case 2:
                    addProduct();
                    break;
                case 3:
                    restockProduct();
                    break;
                case 4:
                    listUsers();
                    break;
                case 5:
                    addUser();
                    break;
                case 6:
```

```
                printOrders(store.getOrders());
                break;
            case 0:
                loggedIn = false;
                break;
            default:
                System.out.println("Unknown option.");
        }
    }
}

private void addProduct() {
    System.out.println("Product type: 1. Book  2. Electronics  3. Clothing");
    int type = input.readInt("Choose: ");
    if (type < 1 || type > 3) {
        System.out.println("Unknown product type.");
        return;
    }
    String name = input.readNonEmpty("Name: ");
    double price = input.readNonNegativeDouble("Price (KZT): ");
    int stock = input.readPositiveInt("Stock quantity: ");
    int id = store.generateProductId();

    Product product;
    if (type == 1) {
        String author = input.readNonEmpty("Author: ");
        int pages = input.readPositiveInt("Pages: ");
        product = new Book(id, name, price, stock, author, pages);
    } else if (type == 2) {
        String brand = input.readNonEmpty("Brand: ");
        int warranty = input.readPositiveInt("Warranty (months): ");
        product = new Electronics(id, name, price, stock, brand, warranty);
    } else {
        String size = input.readNonEmpty("Size: ");
        String color = input.readNonEmpty("Color: ");
        product = new Clothing(id, name, price, stock, size, color);
    }
    store.addProduct(product);
    System.out.println("Product added:");
    product.displayInfo();
}

private void restockProduct() {
    listProducts();
    int id = input.readInt("Product ID: ");
    Product product = store.findProductById(id);
    if (product == null) {
        System.out.println("No product with ID " + id + ".");
        return;
    }
    int quantity = input.readPositiveInt("Quantity to add: ");
    product.increaseStock(quantity);
    System.out.println("New stock for " + product.getName() + ": " + product.getStockQuantity());
}

private void addUser() {
    System.out.println("User type: 1. Customer  2. Employee");
    int type = input.readInt("Choose: ");
    try {
        if (type == 1) {
            store.addUser(readNewCustomer());
        } else if (type == 2) {
            String name = input.readNonEmpty("Name: ");
            String email = input.readNonEmpty("Email: ");
            String password = input.readNonEmpty("Password: ");
            String position = input.readNonEmpty("Position: ");
            double salary = input.readNonNegativeDouble("Salary (KZT): ");
```

```
                store.addUser(new Employee(store.generateUserId(), name, email, password, position,
salary));
            } else {
                System.out.println("Unknown user type.");
                return;
            }
            System.out.println("User added.");
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }

    // ==================== SHARED HELPERS ====================

    private Customer readNewCustomer() {
        String name = input.readNonEmpty("Name: ");
        String email = input.readNonEmpty("Email: ");
        String password = input.readNonEmpty("Password: ");
        String address = input.readNonEmpty("Shipping address: ");
        return new Customer(store.generateUserId(), name, email, password, address);
    }

    private void listProducts() {
        List<Product> products = store.getProducts();
        if (products.isEmpty()) {
            System.out.println("No products available.");
            return;
        }
        System.out.println("--- Products ---");
        for (Product product : products) {
            product.displayInfo(); // polymorphism: each subclass supplies its own details
        }
    }

    private void listUsers() {
        System.out.println("--- Users ---");
        for (User user : store.getUsers()) {
            user.displayInfo();
        }
    }

    private void printOrders(List<Order> orders) {
        if (orders.isEmpty()) {
            System.out.println("No orders yet.");
            return;
        }
        for (Order order : orders) {
            order.printReceipt();
        }
    }
}
```